Universidad Nacional del Nordeste.

Facultad de Ciencias Exactas y Naturales y Agrimensura.

Cátedra: Sistemas Operativos.

Año: 2026.

Grupo N°: 29.

Integrantes:

- Romero Enzo Fabian DNI: 41948148
- Vega Jose Maria DNI:36548306
- Navarro de Jesus DNI: 44683191
- Silva Armand Alejandro Omar DNI: 36774763
- López Santiago Nicolas DNI: 42811857

TEMA: Memoria Virtual.

ÍNDICE:

1.INTRODUCION 4
2.DESARROLLO 5
2.1 Hardware y estructuras de control 5
2.1.1 Proximidad y memoria virtual 6
2.1.2 Paginación 7
2.1.3 Estructura de la tabla de páginas 7
2.1.4 Tabla de páginas invertidas 8
2.1. 5 Buffer de traducción anticipada 9
2.1.6 Tamaño de pagina 11
2.1.7 Segmentación 14
2.1.8 Organización 15
2.1.8 Paginación y segmentación combinada 16
2.1.9 Protección y Compartición 18
2.1.9 Esquema de Protección en Anillos 19
2.2 Software del Sistema Operativo 19
2.2.1 Memoria Virtual y Esquemas de Soporte 19
2.2.2 Políticas de Gestión de Páginas 19
2.2.3 Política de Recuperación 20
2.2.4 Política de Ubicación 20
2.2.5 Política de Reemplazo 21
2.2.6 El Algoritmo del Reloj 21
2.2.7 Estrategias de Buffering de Páginas 24
2.2. 8 gestión del conjunto residente 25
2.2.9 Ámbito de reemplazo 27
2.2.10 Asignación fija, ámbito local 27
2.2.12 Asignación variable, ámbito local 28
2.2.13 Política de limpieza 34
2.2.14 Control de carga 35
2.2.15 Grado de multiprogramación 35
2.2.16 Suspensión de procesos 36
2.3 Gestión de memoria de Unix y solaris 37
2.3.1 Sistema de paginación 37
2.3.2 Gestión de memoria en Linux 42
2.3.3 Memoria virtual en Linux 42
2.3.4 Reserva de memoria del núcleo 43
2.4 Gestión de memoria en Windows 43

2.4.1 Mapa de direcciones virtuales en Windows.....44
2.4.2 Paginación en Windows 44
3. CONCLUSIÓN 46
4. REFERENCIAS BIBLIOGRÁFICAS 46

1.INTRODUCION

W. Stallings [1] describe la memoria virtual como una interacción compleja entre el hardware y el sistema operativo, esencial para la gestión de los recursos de memoria

en sistemas modernos. La gestión de la memoria es fundamental para el funcionamiento de los sistemas operativos modernos. La memoria virtual es un servicio que permite a los programas direccionar la memoria desde un punto de vista lógico, sin depender del tamaño de la memoria física disponible. Durante la ejecución de un programa, solo una parte de sus instrucciones y datos reside en la memoria principal, mientras que el resto se almacena temporalmente en el disco.

En esta monografía, abordaremos en profundidad el funcionamiento de la memoria virtual, sus principales componentes y mecanismos, así como las ventajas que ofrece en la administración eficiente de los recursos del sistema.

2.DESARROLLO

2.1 Hardware y estructuras de control

Comparando la paginación y la segmentación sencillas, por un lado, tenemos una distinción entre particionamiento estático y dinámico, y por otro, tenemos los fundamentos de comienzo de la gestión de la memoria. Las dos características de la paginación y la segmentación que son la clave de este comienzo son:

1) Todas las referencias a la memoria dentro de un proceso s realizan a direcciones lógicas. Esto significa un proceso puede ser llevado y traído a memoria de forma que ocupe diferentes regiones de la memoria principal en distintos instantes de tiempo durante su ejecución.
2) Un proceso puede dividirse en varias porciones (paginas o segmentos), que no tienen que estar localizadas en la memoria de forma contigua durante la ejecución.

Si las dos características anteriores se dan entonces es necesario que todas las páginas o todos los segmentos de un proceso se encuentran en la memoria principal durante la ejecución. los segmentos de un proceso se encuentren en la memoria principal durante la ejecución. Si la porción (segmento o página) en la que se encuentra la siguiente instrucción a buscar está y si la porción donde se encuentra la siguiente dirección de datos que se va a acceder también está, entonces al menos la siguiente instrucción se podrá ejecutar.

1) Pueden mantenerse un mayor número de procesos en memoria principal. Esto permite una utilización más eficiente del procesador porque es más probable que haya al menos uno o más de los numerosos procesos que se encuentre en el estado.
2) Un proceso puede ser mayor que toda la memoria virtual. Con la memoria virtual basada en paginación o segmentación, este trabajo delega al sistema operativo y al hardware.

Debdo a que un proceso ejecutan solo en la memoria principal, esta memoria se denomina memoria real. Pero el programador o el usuario perciben una memoria potencialmente mucho más grande, que se denomina memoria virtual. La memoria virtual permite una multiprogramación muy efectiva que libera al usuario de las restricciones excesivamente fuertes de la memoria principal.

La Tabla 1 recoge las características de la paginación y la segmentación, con y sin el uso de la memoria virtual.

Paginación sencilla	Paginación con memoria virtual	Segmentación sencilla	Segmentación con memoria virtual
Memoria principal particionada en fragmentos pequeños de un tamaño fijo llamados marcos.	Memoria principal particionada en fragmentos pequeños de un tamaño fijo llamados marcos.	Memoria principal no particionada.	Memoria principal no particionada.
Programa dividido en páginas por el compilador o el sistema de gestión de la memoria.	Programa dividido en páginas por el compilador o el sistema de gestión de la memoria.	Los segmentos de programa se especifican por el programador o el compilador (por ejemplo, la decisión se toma por parte del programador).	Los segmentos de programa se especifican por el programador o el compilador (por ejemplo, la decisión se toma por parte del programador).
Fragmentación interna dentro de los marcos.	Fragmentación interna dentro de los marcos.	Sin fragmentación interna.	Sin fragmentación interna.
Sin fragmentación externa.	Sin fragmentación externa.	Fragmentación externa.	Fragmentación externa.
El sistema operativo debe mantener una tabla de páginas por cada proceso, mostrando en qué marco se encuentra cada página ocupada.	El sistema operativo debe mantener una tabla de páginas por cada proceso, mostrando en qué marco se encuentra cada página ocupada.	El sistema operativo debe mantener una tabla de segmentos por cada proceso, mostrando la dirección de carga y la longitud de cada segmento.	El sistema operativo debe mantener una tabla de segmentos por cada proceso, mostrando la dirección de carga y la longitud de cada segmento.
El sistema operativo debe mantener una lista de marcos libres.	El sistema operativo debe mantener una lista de marcos libres.	El sistema operativo debe mantener una lista de huecos en la memoria principal.	El sistema operativo debe mantener una lista de huecos en la memoria principal.
El procesador utiliza el número de página y desplazamiento para calcular direcciones absolutas.	El procesador utiliza el número de página y desplazamiento para calcular direcciones absolutas.	El procesador utiliza el número de segmento y desplazamiento para calcular direcciones absolutas.	El procesador utiliza el número de segmento y desplazamiento para calcular direcciones absolutas.
Todas las páginas del proceso deben encontrarse en la memoria principal para que el proceso se pueda ejecutar, salvo que se utilicen solapamientos (*overlays*).	No se necesita mantener todas las páginas del proceso en los marcos de la memoria principal para que el proceso se ejecute. Las páginas se pueden leer bajo demanda.	Todos los segmentos del proceso deben encontrarse en la memoria principal para que el proceso se pueda ejecutar, salvo que se utilicen solapamientos (*overlays*).	No se necesitan mantener todos los segmentos del proceso en la memoria principal para que el proceso se ejecute. Los segmentos se pueden leer bajo demanda.
–	La lectura de una página a memoria principal puede requerir la escritura de una página a disco.	–	La lectura de un segmento a memoria principal puede requerir la escritura de uno o más segmentos a disco.

Tabla 1: Características de la paginación y la segmentación

2.1.1 Proximidad y memoria virtual

La memoria virtual, basada en paginación o paginación más segmentación, se ha convertido, en la actualidad, en una componente esencial de todos los sistemas operativos contemporáneos.

El principio de proximidad indica que las referencias al programa y a los datos dentro de un proceso tienden a agruparse. Por lo tanto, se resume que solo unas pocas porciones del proceso necesitaran a lo largo de un periodo de tiempo corto. También, es posible hacer suposiciones inteligentes sobre cuáles son las porciones del proceso que se necesitarán en un futuro próximo, para evitar este trasiego.

Para que la memoria virtual resulte practica y efectiva se necesitan dos ingredientes:

1) La existencia de un hardware para el esquema de paginación y/o segmentación.

2) El sistema operativo debe incluir código para gestionar el movimiento de paginación y/o segmentos- entre la memoria secundario y la memoria principal.

2.1.2 Paginación

El termino memoria virtual se asocia habitualmente con sistemas que emplean paginación, a pesar de que la memoria virtual basada en segmentación se utiliza. El uso de paginación para conseguir memoria virtual utilizado por primera vez en el computador de Atlas [KILB62] y pronto se convirtió en una estrategia usada en general de forma comercial.

Para la memoria virtual basada en el esquema de paginación se necesita una tabla de página. Normalmente se asocia a una única tabla de página a cada proceso, pero en este caso las entradas de la tabla de página son más completas. Debía a que solo algunas de las páginas de proceso se encuentran en la memoria principal, se necesita que cada entrada de la tabla de página indique si la correspondiente página está presente en la memoria principal o no. Si el bit indica que la página está en la memoria, la entrada también debe indicar el número de marco de dicha página

La entrada de la tabla de páginas incluye un bit de modificado, que indica si los contenidos de la correspondiente página han sido alterados desde que la página se cargó por última vez en la memoria principal. si no había ningún cambio, no es necesario escribir la página cuando llegue el momento de reemplazarla por otra página en el marco de página que actualmente ocupa. Puede existir también otros bits de control en estas entradas.

2.1.3 Estructura de la tabla de páginas

El mecanismo básico de lectura de una palabra de la memoria implica la traducción de la memoria virtual, o lógica, consiste en el número de páginas y un desplazamiento, a la dirección física, concise en un numero de marco y un desplazamiento, usando para ello la tabla de páginas.

La tabla de páginas es de longitud variable dependiendo del tamaño del proceso, no se puede suponer que se encuentra almacenado en los registros, en lugar de eso, se debe encontrar en la memoria principal para poder ser accedida

Cuando un proceso en particular se encuentra ejecutando, un registro contiene la dirección de comienzo de las tablas de páginas para dichos procesos. El número de páginas de la dirección virtual se utiliza para indexar esa tabla y buscar el correspondiente marco de páginas. Normalmente el campo correspondiente al número de página es mayor que el campo correspondiente al número de marco de página.

2.1.4 Tabla de páginas invertidas

Una desventaja de los tipos de tablas de páginas que hemos visto es que su tamaño es proporcional al espacio de direcciones virtuales.

Una estrategia alternativa al uso de tablas de páginas de uno o varios niveles es la tabla de páginas invertida.

La figura 1 muestra una implementación típica de esta técnica. Para un tamaño de memoria física de 2m2^m2m marcos, la tabla de páginas invertida contiene 2m2^m2m entradas, de modo que la entrada en la posición *i-ésima* se refiere al marco *i*.

Cada entrada en la tabla de páginas incluye la siguiente información:

- Número de página: Parte correspondiente al número de página dentro de la dirección virtual.
- Identificador del proceso: Indica el proceso propietario de esa página. La combinación del número de página y el identificador del proceso identifica de forma única una página dentro del espacio de direcciones virtuales de un proceso determinado.
- Bits de control: Este campo incluye los flags o indicadores de estado asociados a la página.
- **Puntero de cadena:** Este campo es nulo si no hay más entradas encadenadas a esta. En caso contrario, contiene el índice de la siguiente entrada en la cadena.

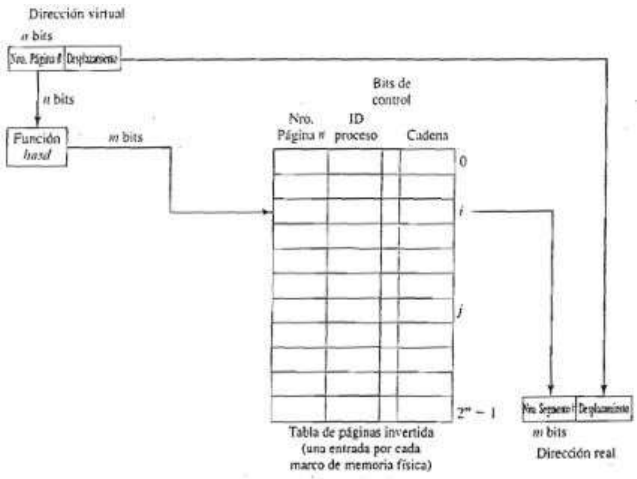


Figura 1: Estructura de tabla de páginas invertida.

2.1. 5 Buffer de traducción anticipada

Toda referencia anticipada a la memoria virtual puede causar dos accesos a la memoria física: uno para buscar la entrada correspondiente en la tabla de página, y otro para buscar los datos solicitados. En consecuencia, en el esquema virtual básico, el tiempo de acceso a la memoria se duplica.

Para mitigar este problema, en la mayoría de los esquemas de memorias virtuales se utilizan memorias *caches* de alta velocidad, destinadas a acelerar el acceso a la tabla de páginas, denominadas *buffer de traducción anticipada (translación lookaside buffer-TLB). Esta* cache contiene las entradas de página de uso más reciente. La organización de hardware resultante de paginación se muestra en la Figura 2.

Dado una memoria virtual el procesador busca primero en la TLB. Si la entrada a la página está presente (*acierto en TLB*), se recupera el número de marco y se construye la dirección real. En caso contrario (*fallo en la TLB*), el procesador utiliza el número de páginas para indexar la tabla de páginas del proceso y examinar las correspondientes entradas.

Si el bit de presente se halla en 1, significa que el número de páginas se encuentra en memoria principal, por lo que puede recuperarse el número de marco desde la entrada de tabla de página para construir la dirección real. En este caso el procesador autorizara a la TLB, incluir esta nueva entrada de tabla de página. Si por el contrario el bit de presente no se halla en 1, la página solicitada no se halla en la memoria principal y se produce un fallo de acceso a memoria, denominado fallo de página. Ahora abandonamos el dominio del hardware, para invocar al sistema operativo, el mismo cargara la página necesaria y actualizada de la tabla de páginas.

La Figura 3 muestra un diagrama de flujo de uso de la TLB. En él se observa que, si una página solicitada no se encuentra en la memoria principal, se produce un fallo de página, lo que provoca la invocación de la rutina de su tratamiento. Gracias al principio de proximidad, la mayoría de referencias a la memoria virtual se encontrarán en una página recientemente utilizada y por lo que la mayoría de referencias invocaran una entrada de la tabla de páginas que se encuentran en la *cache***.** Los estudios sobre la TLB de los sistemas VAX, demostraron que este esquema representa una mejora importante en el rendimiento.

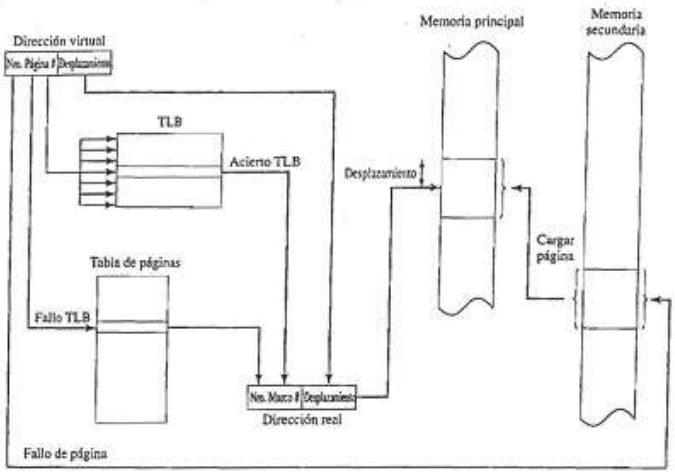


Figura 2: Uso de la TLB

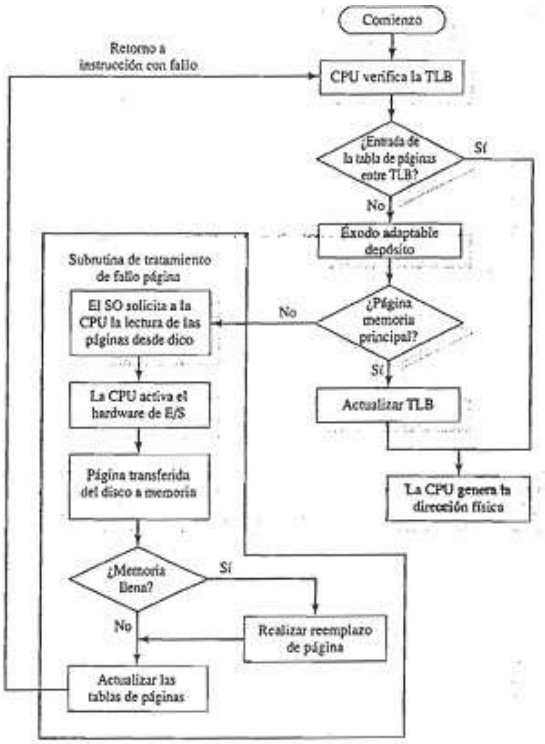


Figura 3: Operación de paginación y TLB

Debido a que la TLB contiene algunas de las entradas de la tabla de páginas, no es posible indexarla directamente usando el número de página. En su lugar, cada entrada de la TLB debe incluir un numero de página, así como la tabla de páginas completa. El procesador proporciona un hardware que permite consultar simultáneamente varias entradas, para determinar si alguna coincide con el número de páginas buscado. Esta técnica se denomina resolución asociativa (*assosiative mapping*), en contraste con la indexación utilizada para buscar en la tabla de página Figura 4.

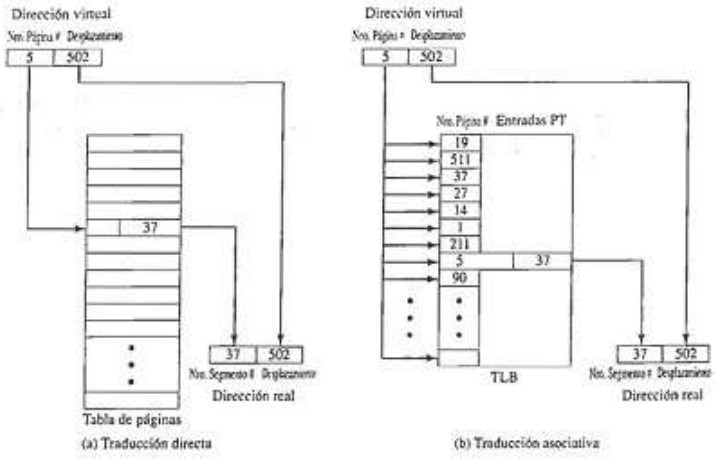


Figura 4: Resolución directa vs. asociativa para las entradas en la tabla de las páginas.

Además, se deberán tener en cuenta temas de diseño de la TLB, como se organizan las entradas en ella, y que entrada reemplazar, cuando se solicite nuevas entradas.

Concluyendo, el mecanismo de memoria virtual debe interactuar con el sistema cache (no la cache TLB, sino la cache de la memoria principal). Como se ilustra Figura 6 Donde una dirección virtual tendrá formato de página número de página más desplazamiento. Siendo que Primero el sistema de memoria consulta a la TLB para ver si se encuentra una entrada de la tabla de página que coincida. Si es así, la dirección real (*física*) se genera, combinando el número de marco con el desplazamiento. Si no, la entrada busca en la tabla de páginas. Una vez generado la dirección real, que mantiene el formato de etiqueta (tag) y resto (remainder), se consulta la cache para ver si el bloque que contiene esa palabra se halla ahí. Si es así se le devuelve a la CPU. Si no, la palabra se busca en la memoria principal.

El acceso a memoria por parte de la CPU constituye un proceso de notable complejidad. La dirección virtual generada por el programa debe traducirse a una dirección real mediante la tabla de páginas, cuya entrada puede encontrarse en la TLB, en la memoria principal o en el disco. Si dicha palabra se halla en el disco la página que la contiene debe cargarse en la memoria principal junto con su bloque correspondiente en la cache. Además, la entrada en la tabla de páginas para dicha página debe ser actualizada.

2.1.6 Tamaño de pagina

En el diseño de hardware, una decisión fundamental es la elección del tamaño de página a utilizar. Existen varios factores que deben considerarse:

1. *Fragmentación interna*: A mayor tamaño de página, menor cantidad de fragmentación interna.

2. *Número de páginas*: Cuanto menor es el tamaño de la página, mayor será el número de páginas necesarias por proceso, lo que implica la utilización de tablas de páginas más grandes.

Para programas de gran tamaño en entornos multiprogramados, esto significa, que determinadas partes de la tabla de página de los procesos activos deben encontrarse en la memoria virtual, no en la memoria principal. Por lo tanto, puede producirse un fallo de página doble, que implicaría dos situaciones. Uno para traer la tabla de páginas de la parte solicitada y otro para traer la tabla del propio proceso.

Otro factor importante son las características físicas de la mayoría de los dispositivos de memoria secundaria del tipo giratorio **(***discos duros magnéticos HDD*), los cuales favorecen el uso de tamaño de página grandes, mejorando la eficiencia en la transferencia de bloques de datos.

Si se aumentan la complejidad de los aspectos mencionados, se halla el efecto, donde el tamaño de página relaciona con que ocurra un fallo de página, comportamiento observado en Figura 6, basado en el principio de proximidad.

Si el tamaño, de cada página es pequeño, habrá un número relativamente alto de páginas disponibles en la memoria principal para cada proceso. Transcurrido un tiempo, las páginas en memoria, contendrán parte de ese proceso referenciados recientemente, por lo que la tasa de fallos debería ser baja. A medida que el tamaño de las páginas aumenta de tamaño cada página, contendrá información más alejada de la última referencia realizada. En consecuencia, el principio de proximidad se debilita y la tasa de fallos comienza a crecer. En cierto punto la tasa de fallos comienza a caer a medida que el tamaño de la página se aproxima al tamaño del proceso completo (punto P diagrama). Cuando una única página contiene el proceso completo, no habrá fallos de página.

Una complicación adicional es que la tasa de fallos de página también depende del número de marcos asignados a cada proceso. La Figura 6 muestra que, para un tamaño de páginas fijo, la tasa de fallos disminuye a medida que aumenta el número de páginas mantenidas en la memoria principal. Por lo tanto, la cantidad de memoria reservada para cada proceso (política de software) interactúa con decisiones de diseño del hardware (como el tamaño de página).

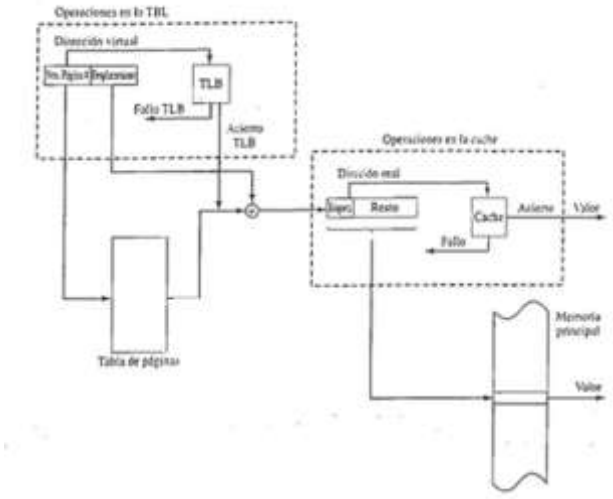


Figura 5: Operación en la TLB y la cache.

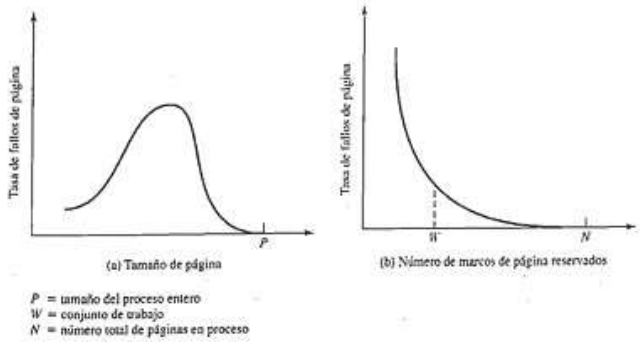


Figura 6: Comportamiento típico de la paginación de un programa.

Concluyendo, el aspecto en el diseño de página se encuentra relacionado con el tamaño de la memoria física y el tamaño de programas. Al mismo tiempo que la memoria principal se vuelve cada vez más grande, también crece el espacio de direcciones utilizado por las aplicaciones, especialmente en ordenadores con programas cada vez más complejos. Sin embargo, las técnicas para manejo de programas de gran tamaño reducen el efecto de proximidad dentro de un proceso.

Ejemplo:

1. *Técnicas de programación orientada a objetos:* Fomentan el uso de numerosos módulos de datos y programas de menor tamaño, con referencias distribuidas entre un gran número de objetos en un corto lapso.
2. *Aplicaciones Multihilo (multithread):* Pueden presentar o no, flujo de instrucciones y referencias a memoria fraccionadas.

Para un tamaño determinado de una TLB, a medida que el proceso aumenta en tamaño y la proximidad de referencia disminuye, el índice de aciertos en la TLB se reduce, generando un problema de rendimiento.

Una forma de aumentar dicho rendimiento es utilizar una TLB de mayor tamaño, con más entradas. Sin embargo, el tamaño de la TLB interactúa con otros aspectos de diseño de hardware, como la cache de memoria principal; número de acceso a memoria por ciclo de instrucción.

Además, existe una diferencia en la velocidad de crecimiento entre el tamaño de la TLB y el de la memoria principal. Como alternativa se utilizan páginas de mayor tamaño, de forma que cada entrada en la tabla de páginas referenciada en la TLB apunte a un bloque de memoria grande.

Siendo esto una contradicción sobre la utilización de páginas de gran tamaño los cuales significan perdida de rendimiento.

Computer	Tamaño de página
Atlas	512 palabras de 48-bits
Honeywell-Multics	1024 palabras de 36-bits
IBM 370/XA y 370/ESA	4 Kbytes
Familia VAX	512 bytes
IBM AS/400	512 bytes
DEC Alpha	8 Kbytes
MIPS	4 Kbytes hasta 16 Mbytes
UltraSPARC	8 Kbytes hasta 4 Mbytes
Pentium	4 Kbytes o 4 Mbytes
PowerPc	4 Kbytes
Itanium	4 Kbytes hasta 256 Mbytes

Tabla 2: Ejemplo de tamaño de páginas.

Con base a las consideraciones anteriores, se ha investigado el uso de múltiples tamaños de páginas, y diferentes arquitecturas de microprocesadores (MIPS, R 4000, Alpha, UltraSPARC, PENTHIUM, IA64) brindan soporte para ellos. Esto proporciona mayor flexibilidad y eficiencia en el uso de la TLB.

En las regiones contiguas grandes de memoria, dentro del espacio de direcciones del proceso, como las instrucciones de programa, es posible mapear sobre un número reducido de páginas grandes, en lugar de un gran número de páginas más pequeñas, mientras que las pilas de los distintos hilos pueden alojarse en páginas de menor tamaño. Esto se debe a que el tamaño de página influye directamente en el funcionamiento del sistema operativo.

2.1.7 Segmentación

Las implicaciones en la memoria virtual**.** La segmentación permite al programador observar la memoria como distintos espacios de direcciones de diferentes tamaños.

Una referencia a la memoria adopta el formato de direcciones del tipo *número de segmento, desplazamiento.*

Esta organización ofrece ventajas al programador frente a los espacios de direcciones no segmentadas:

1. Simplifica el tratamiento de estructuras de datos que puedan crecer. Si el programador no conoce anticipadamente el tamaño que dicha estructura puede alcanzar, es necesario realizar una estimación, salvo que se utilizan tamaño de segmento dinámico. En una memoria virtual segmentada, a una estructura de datos se le puede asignar su propio segmento, y el sistema operativo podrá reducir o expandir dicho segmento bajo demanda.

   En caso de que un segmento necesite expandirse y se encuentre en la memoria principal, sin suficiente espacio, el sistema operativo podrá moverlo a un área disponible de memoria principal mayor o enviarlo a SWAP.
2. Permite que los programas se modifiquen o recopilen independientemente. Sin requerir que todo el programa se reenlace y vuelva a cargarse. Esta posibilidad se logra mediante el uso de múltiples segmentos.
3. Da soporte a la compartición entre procesos. El programador puede ubicar un programa o tabla de datos útil en un segmento, al que pueda referenciar desde otro proceso.
4. Soporta los mecanismos de protección, ya que un segmento puede definirse para contener un conjunto de programas o datos bien estructurados, y el programador o administrador del sistema puede asignar privilegios de acceso de forma adecuada.

2.1.8 Organización

En la segmentación sencilla, cada proceso tiene su tabla de segmentos. Una vez que todos los segmentos del proceso se cargan en la memoria principal, se genera la tabla de segmentos correspondiente y se almacenara en dicha memoria. Cada entrada de la tabla de segmentos contiene la dirección de inicio del segmento en la memoria principal y su longitud.

Este mismo mecanismo se utiliza al tratar esquemas de memoria virtual basados en segmentación. habitualmente, existe una única tabla de segmentos por proceso, aunque en algunos casos las entradas de dichas tablas, son más complejas, ya que solo ciertos segmentos de procesos pueden encontrarse en la memoria principal. Para ello se requiere un bit en cada entrada de dicha tabla que indique si el correspondiente segmento se encuentra o no en la memoria principal.

Si el segmento se encuentra (*En memoria principal*), la entrada debe incluir su dirección de comienzo y su longitud.

El otro bit de control en la entrada de la tabla de segmento es el bit modificado, que indica si el contenido del segmento se ha modificado desde la última vez que fue cargado en la memoria principal.

El mecanismo básico para la lectura de una palabra de memoria implica la traducción de una memoria virtual o lógica, consistente en un numero de segmento y desplazamiento, en una dirección física, usando la tabla se segmentos.

Debido a que dicha tabla tiene un tamaño variable, dependiendo del tamaño del proceso, no puede suponerse que se halle almacenada en un registro; por lo tanto, debe encontrarse en la memoria principal para su acceso.

La Figura 7, propone un hardware con este esquema.

Cuando un proceso particular está en ejecución, un registro mantiene la dirección de comienzo de la tabla de segmentos para dicho proceso.

El número de segmento de la dirección se utiliza para indexar esta tabla y buscar la dirección de la memoria principal donde comienza dicho segmento. Esta se añade a la parte de desplazamiento de la dirección virtual para producir la dirección real solicitada.

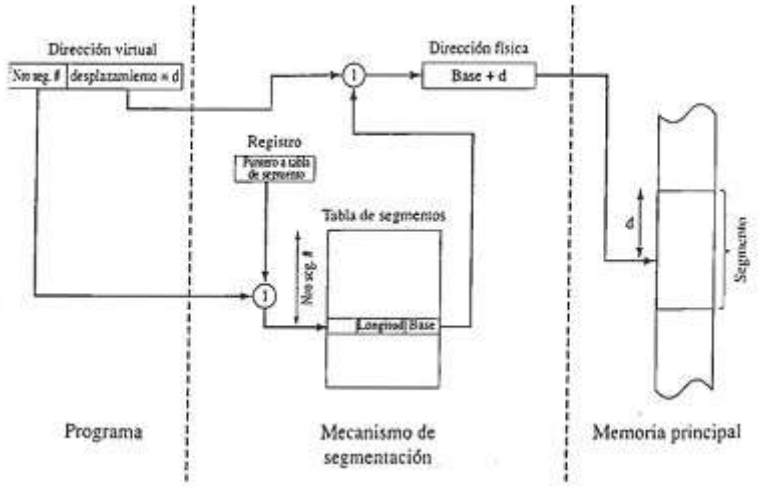


Figura 7: Traducción de direcciones en un sistema con segmentación.

2.1.8 Paginación y segmentación combinada

Cada una de estas tiene ventajas particulares.

La paginación es transparente al programador y elimina la fragmentación externa, proporcionando un uso eficiente de la memoria principal. Además, los fragmentos que se mueven entre la memoria y el disco son de un tamaño igual y prefijado, lo que permite

desarrollar algoritmos de la memoria más sofisticados que mejoren el desempeño de los programas.

Por su parte la segmentación ofrece ventajas como el manejo de estructuras de datos que crecen, modularidad, y soporte a compartición y protección.

Para combinar las ventajas de ambos métodos, algunos sistemas, mediante el hardware de procesador y el soporte del sistema operativo, son capaces de implementarlos conjuntamente.

En un sistema combinado de paginación y segmentación, el espacio de direcciones del usuario se divide en un numero de segmentos, a criterio del programador.

Cada segmento se divide en un número de páginas de tamaño fijo, iguales al tamaño de los marcos de la memoria principal. Si la longitud de un segmento es inferior a una página, el segmento ocupara únicamente una página.

Desde el punto de vista del programador, una dirección lógica contiene un numero de segmento y un desplazamiento dentro del mismo.

Desde el punto de vista del sistema, el desplazamiento dentro del segmento se observa como un numero de página, y un desplazamiento dentro de la página, incluida el segmento.

La Figura 8 indica soporte de paginación y segmentación. Cada proceso tiene una tabla de segmentos y, para cada segmento, una tabla de páginas. Cuando el proceso se ejecuta, se mantiene en un registro la dirección de las tablas de segmento.

A partir de la dirección virtual, el procesador usa el número de segmento para localizar la tabla correspondiente y así acceder a la tabla de páginas de dicho segmento. Una vez identificado el segmento, el número de página de la dirección virtual se usa para buscar el número de marco correspondiente en la tabla de páginas. Luego, este número de marco se combina con el desplazamiento de la dirección virtual para obtener la dirección física o real en memoria.

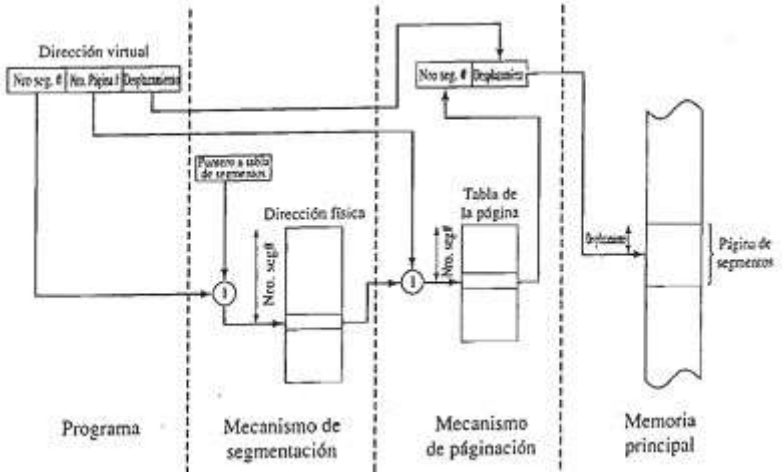


Figura 8: Traducción de direcciones en un sistema con segmentación/paginación.

En la Figura 6, se muestran los formatos de las entradas de la tabla de segmentos y de la tabla de páginas. La primera contiene la longitud del segmento, además del campo base, que hace referencia a la tabla de páginas. Los bits presente y modificado no se necesitan, aunque si pueden existir otros bits de control. En cuanto a la segunda, la entrada de página es similar a la del sistema de paginación pura. El número de página se proyecta en un numero de marco cuando se encuentra presente en memoria. El bit modificado indica si la página necesita escribirse cuando sea expulsado del marco actual.

2.1.9 Protección y Compartición

La segmentación es un mecanismo clave para implementar políticas de protección y compartición de memoria.

La segmentación como base no permite que un programa, pueda acceder de forma descontrolada a una posición de memoria principal más allá de los límites de su segmento, ya que la entrada en la tabla de segmentos incluye la longitud y la dirección base. Para conseguir esta compartición es posible que un segmento este referenciado en la tabla de segmentos de varios procesos, estos mecanismos están disponibles en los sistemas de paginación. Aunque este sistema y sus datos no serán visibles para el programador haciendo menos cómoda la gestión de protección y compartición.

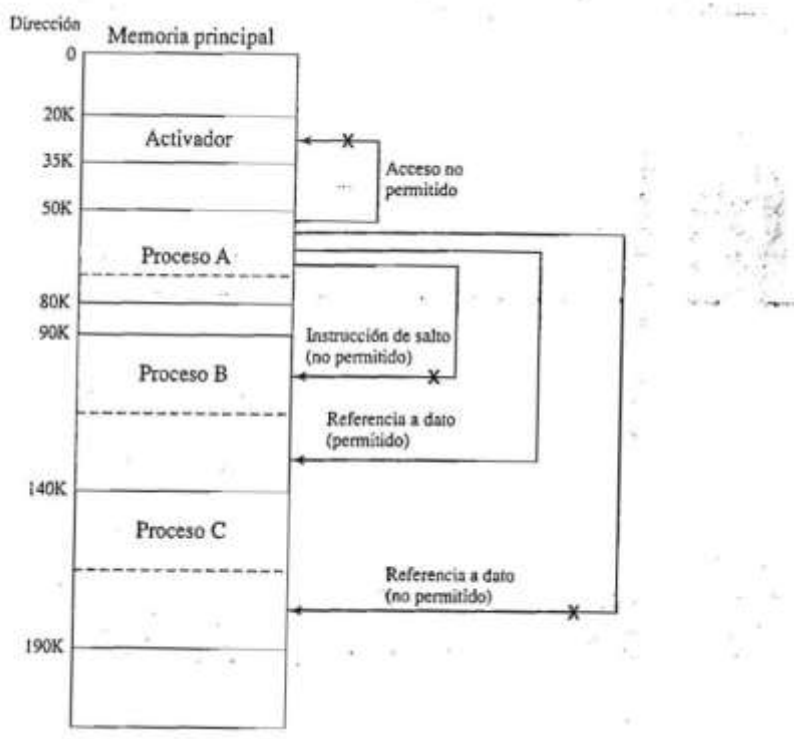


Figura 9 : relaciones de proteccion entre segmentos

2.1.9 Esquema de Protección en Anillos

Un mecanismo más sofisticado es la estructura de protección en anillo.

- Los anillos con numeraciones más bajas o interiores disfrutan de mayores privilegios que los anillos con numeraciones más altas o exteriores.
- Normalmente, el anillo 0 se reserva para las funciones del núcleo del sistema operativo.
- Las aplicaciones se sitúan en niveles superiores, y algunas utilidades o servicios del S.O. pueden ocupar un anillo intermedio.
- Un programa solo puede acceder a los datos residentes en el mismo anillo o en anillos con menos privilegios.
- Un programa solo puede invocar servicios residentes en el mismo anillo o anillos con más privilegios.

2.2 Software del Sistema Operativo

La gestión de la memoria en los sistemas operativos modernos es un aspecto crítico del rendimiento y la funcionalidad del sistema. Este diseño depende fundamentalmente de tres decisiones clave: la elección de usar o no técnicas de memoria virtual; la implementación de paginación, segmentación, o una combinación de ambas; y los algoritmos específicos para los diversos aspectos de la gestión de memoria.

2.2.1 Memoria Virtual y Esquemas de Soporte

La mayoría de los sistemas operativos importantes en la actualidad proporcionan memoria virtual. El uso de esta técnica requiere una plataforma hardware capaz de realizar la traducción de direcciones, ya que ninguna técnica es abordable sin dicho soporte de hardware.

Siguiendo la idea, desprendemos dos comentarios:

- Todo buen sistema operativo está preparado para administrar memoria virtual.
- Los sistemas de segmentación puros son muy escasos en la actualidad, por lo que se opta por implementar una mezcla de paginación y segmentación. Aunque comúnmente el sistema de archivos se centrará en la paginación, por lo que veremos aspectos muy similares a ésta última.

2.2.2 Políticas de Gestión de Páginas

En los siguientes títulos analizaremos los algoritmos para la gestión de la memoria, la cual estará encargada de brindarle el mejor rendimiento al sistema:

Política de recuperación	**Gestión del conjunto residente**
Bajo demanda	Tamaño del conjunto residente
Paginación adelantada	Fijo
	Variable
Política de ubicación	Ámbito de reemplazo
	Global
Política de reemplazo	Local
Algoritmos básicos	
Óptimo	**Política de limpieza**
FIFO	Bajo demanda
Usada menos recientemente (LRU)	Limpieza adelantada
Del reloj	**Control de carga**
Buffers de página	Grado de multiprogramación

Tabla 3: políticas de sistemas operativos para la memoria virtual

2.2.3 Política de Recuperación

La política de recuperación determina cuándo se trae una página a la memoria principal.

- Paginación por Demanda (Demand Paging): Una página se trae a memoria solo cuando se hace referencia a una posición en dicha página (es decir, ocurre un fallo de página).
- Paginación Adelantada (Prepaging): Se traen a memoria también otras páginas, diferentes de la que ha causado el fallo, basándose en la suposición de que serán necesarias próximamente. Esta es eficiente cuando un proceso arranca inicialmente y es ineficiente si la mayoría de las páginas traídas no se referencian a posteriori. Esta política no debe confundirse con el swapping (intercambio), donde todas las páginas de un proceso son expulsadas cuando se saca de la memoria.

2.2.4 Política de Ubicación

La política de ubicación determina en qué parte de la memoria real van a residir las porciones de la memoria de un proceso.

- Sistemas de Segmentación Pura: La ubicación es un aspecto de diseño importante, utilizando políticas como mejor ajuste, primer ajuste y peor ajuste.
- Sistemas de Paginación o Paginación Combinada: La ubicación es habitualmente irrelevante, ya que el hardware de traducción puede realizar sus funciones con igual eficiencia en cualquier combinación de página-marco.

2.2.5 Política de Reemplazo

La política de reemplazo es la selección de la página en la memoria principal candidata a ser reemplazada cuando se va a traer una nueva página y todos los marcos están ocupados. Su objetivo es que la página eliminada sea aquella que tiene menos posibilidades de volver a tener una referencia en un futuro próximo.

Cabe destacar que mientras más elaborada sea la política de reemplazo, más sobrecarga de software y mayores especificaciones de hardware necesitará.

Los algoritmos de reemplazo básicos incluyen:

- Óptimo: Selecciona la página para la cual el instante de la siguiente referencia se encuentra más lejos. Es imposible de implementar, pero se usa como estándar de contraste.
- Usado menos recientemente (LRU - Least Recently Used): Selecciona la página que no se haya referenciado desde hace más tiempo. Sus resultados son casi tan buenos como la política óptima, pero su implementación es difícil e impone una sobrecarga significativa.
- FIFO (First-in-first-out): Reemplaza la página que lleva más tiempo en memoria. Es muy sencilla de implementar.
- Reloj: Variante de LRU que utiliza un bit de usado (bit adicional en el marco) como política de reemplazo.

Una restricción importante en esta política es el bloqueo de marcos. Los marcos bloqueados no pueden ser reemplazados; gran parte del núcleo del sistema operativo, así como los buffers de E/S, se almacenan en marcos bloqueados.

El contenido se centra en las políticas de reemplazo de páginas y la gestión de memoria virtual en Sistemas Operativos, detallando algoritmos específicos (Óptimo, LRU, FIFO, Reloj) y estrategias de buffering para mejorar el rendimiento.

2.2.6 El Algoritmo del Reloj

El algoritmo del reloj es una variante diseñada para aproximarse a LRU con una sobrecarga reducida. Se encuentra muy próximo en rendimiento al LRU.

Cada marco presenta un bit adicional, el cual si es referenciado se pondrá en 1, caso contrario se pondrá en 0; al momento de reemplazar un marco recorre en forma de reloj todos los marcos, cuando encuentre un marco con el bit en 0, lo selecciona para ser reemplazado, pero si está en 1 lo pondrá en 0 y seguirá con su camino. En caso de que todos los marcos estén

en 1, los pondrá todos en 0 y elegirá el marco que estaba en la posición original, de ahí su semejanza con FIFO.

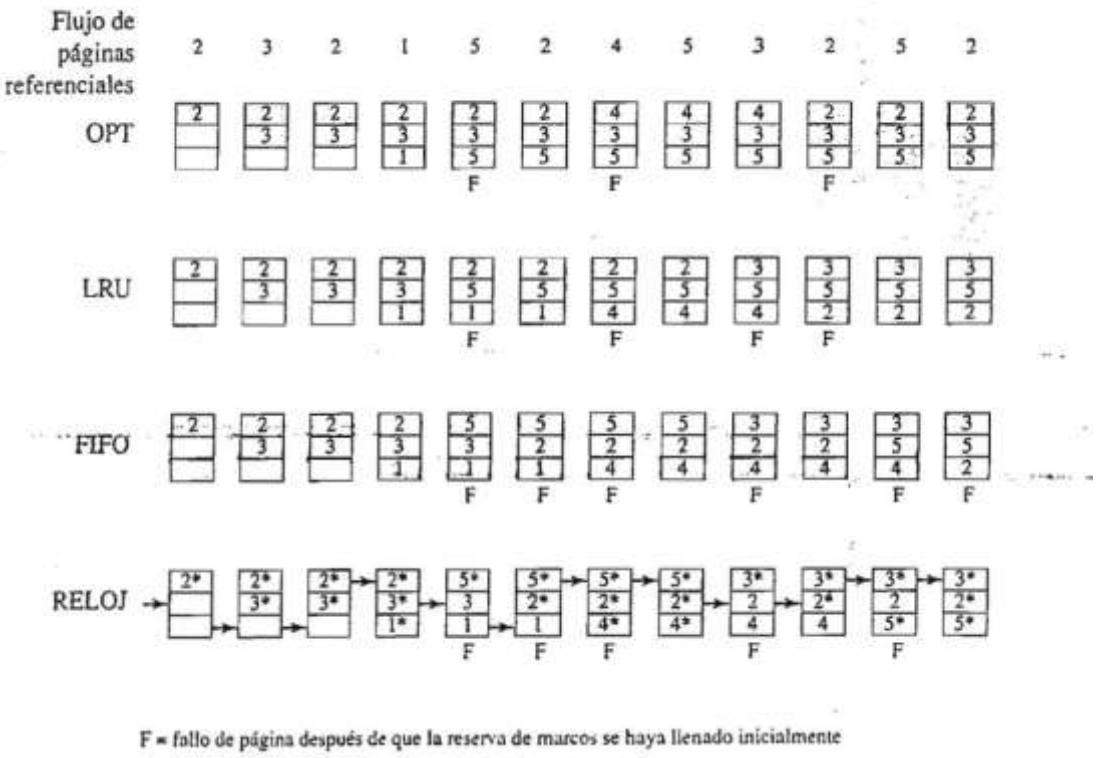


Figura 10: Comportamiento de cuatro algoritmos de reemplazo de paginas

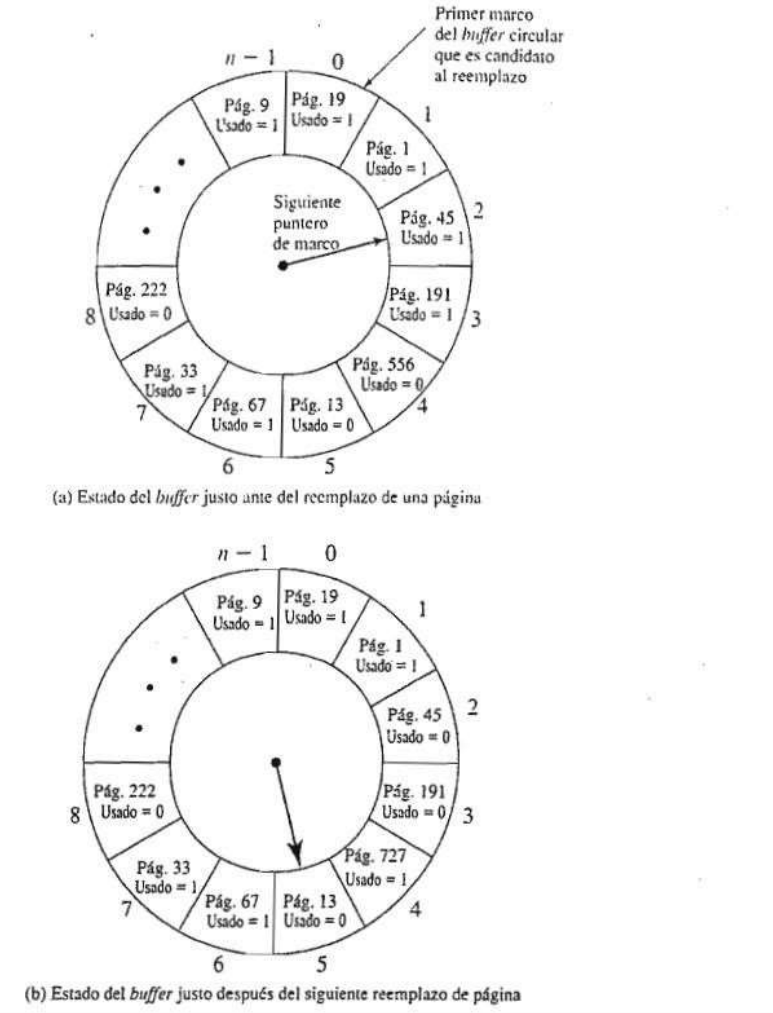


Figura 11: Ejemplo de operación de la política de reemplazo del reloj.

Categorías de Reemplazo (Usado y Modificado)

El algoritmo del reloj se podría hacer más potente aplicando un bit más para “modificado”.

El algoritmo clasifica cada marco en cuatro categorías, utilizando el bit de usado (u) y el bit de modificado (m):

- No accedida recientemente, no modificada (u=0; m=0).
- Accedida recientemente, no modificada (u=1; m=0).
- No accedida recientemente, modificada (u=0; m=1).
- Accedida recientemente, modificada (u=1; m=1).

El algoritmo busca primero un marco con (u=0; m=0) para reemplazar, ya que es la mejor opción para reemplazo porque no se ha modificado y no necesita escribirse en la memoria secundaria. Si no lo encuentra en la primera vuelta, busca otras categorías (u=0; m=1), si no

se encuentran, hace otro recorrido poniendo el bit de usado a 0 en los marcos que salta para darles una segunda oportunidad.

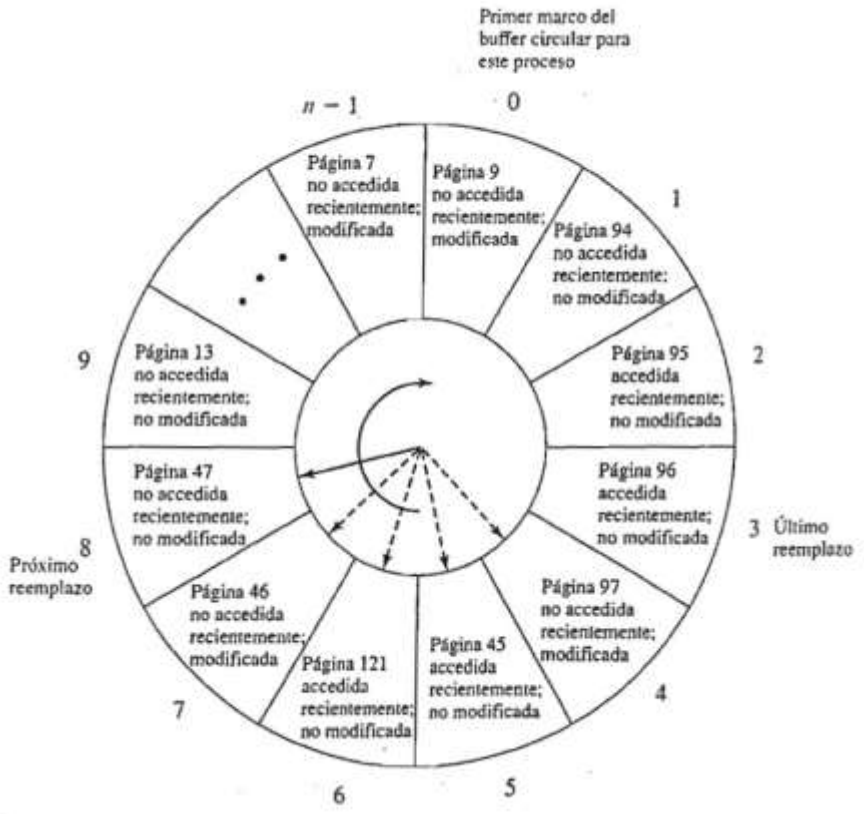


Figura 12: el algoritmo de remplazo de páginas de reloj

2.2.7 Estrategias de Buffering de Páginas

Las políticas LRU y Reloj son superiores a FIFO, pero ambas incluyen complejidad y sobrecarga que FIFO no sufre. El coste de reemplazar una página modificada es superior al de una no modificada, ya que la primera debe escribirse en la memoria secundaria.

Una estrategia para mejorar el rendimiento con una política sencilla (como FIFO) es el buffering de páginas.

Mecanismo VAX VMS: Una página reemplazada no se "pierde", sino que se asigna a una de dos listas:

- Lista de páginas libres: Si la página no se ha modificado.
- Lista de páginas modificadas: Si la página ha sido modificada.

Beneficio: La página reemplazada se mantiene en la memoria. Si el proceso vuelve a referenciarla, se devuelve al conjunto residente con un bajo coste. La lista de páginas

modificadas reduce significativamente el número de operaciones de E/S y el tiempo de acceso a disco, ya que las páginas modificadas se escriben en grupos.

Cache de Páginas: La lista de páginas modificadas y libres actúa como un cache de páginas. El siguiente grafico nos muestra la comparación de los resultados de los algoritmos mencionados, el resultado se basa en la ejecución de 0,25 x 10^6 referencias en un programa en lenguaje FORTRAN, utilizando un tamaño de página de 256 palabras.

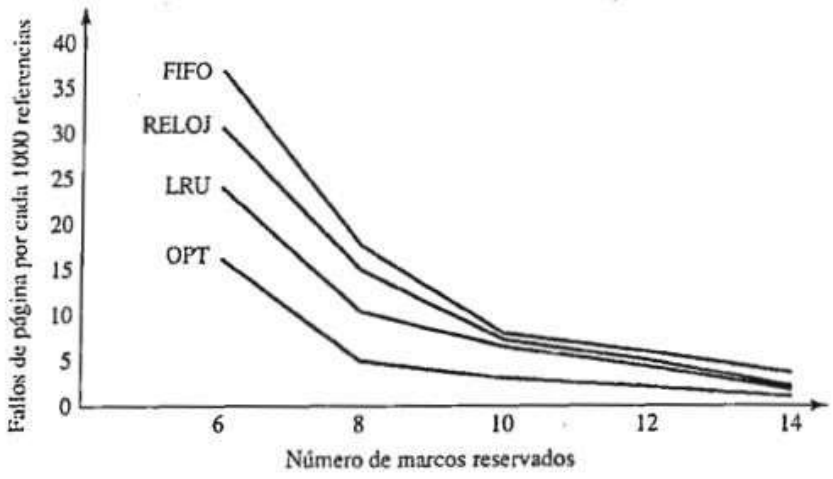


Figura 13: comparativas de algoritmos de remplazo local con reserva de marcos fija.

En sistemas que utilizan algún tipo de buffering de páginas, se puede mejorar el rendimiento de la cache añadiendo a la política de reemplazo de páginas una política de ubicación de páginas en el buffer de páginas. La mayoría de los sistemas operativos ubican las páginas seleccionando un marco procedente del buffer de páginas, de forma arbitraria. El estudio reportado en [KESS92] muestra que una estrategia de reemplazo de páginas cuidadosa puede significar entre un 10 y 20% menos de fallos de cache si se compara con un reemplazo simple.

2.2. 8 gestión del conjunto residente

En algunos casos no es ni siquiera posible, traer todas las páginas de un proceso a la memoria principal para preparar su ejecución. Debido a que el sistema operativo debería

saber cuántas paginas traerse, esto es cuanta memoria principal debería reservar para un proceso en particular. Diferentes factores entran en juego:

- Cuanto menor es la cantidad de memoria reservada para un proceso, mayor es el número de procesos que pueden residir en la memoria principal. Esto aumenta la probabilidad de que el SO (Sistema Operativo) pueda encontrar al menos un proceso listo para ejecutar en un instante dado, así, reduce el tiempo perdido debido al swapping.
- Si el conjunto de páginas de un proceso que está en la memoria es relativamente pequeño, entonces, en virtud del principio de proximidad de referencia, la posibilidad de un fallo de página es mayor.
- Mas allá de un determinado tamaño, la reserva de más memoria principal para un determinado proceso no tendrá un efecto apreciable sobre la tasa de fallos de página de dicho proceso, debido al principio de proximidad de referencia.

Teniendo en cuenta estos factores, se puede encontrar dos tipos de políticas en los sistemas operativos contemporáneos. La política de asignación fija: proporciona un numero fijo de marcos de memoria principal disponibles para ejecución. Este número se decide en el momento de la carga inicial de proceso (instante donde se crea el proceso) y se puede determinar en base al tipo de proceso o se puede basar en las guías proporcionadas por el programador o administrador del sistema. Siempre que se produzca un fallo de página del proceso en ejecución, la página que se necesite reemplazara una de las páginas del proceso.

Una política de asignación variable permite que se reserven un numero de marcos de proceso que puede varias a lo largo del tiempo de vida del mismo. De forma ideal, a un proceso que este causando una tasa de fallos de página relativamente alta de forma continua, indicativo de que el principio de proximidad solo se aplica de forma relativamente débil para estos procesos, se le otorgaran marcos de página adicionales para reducir esta tasa de fallos: mientras tanto, a un proceso con una tasa de fallos excepcionalmente baja, indicativo de que el proceso sigue un comportamiento bien ajustado al principio de proximidad de referencia, se reducirán los marcos reservados, con esperanza de que esto no incremente de forma apreciable la tasa de fallos. El uso de las políticas de asignación variable se basa en el concepto de ámbito de reemplazo.

La política de asignación variable podría parecer más potente. Sin embargo, las dificultades de esta estrategia se deben a que el SO debe saber cuál es el comportamiento del proceso

activo. Esto requiere, de forma inevitable, una sobrecarga sobre el software por parte del SO y depende de los mecanismos de hardware por la propia plataforma del procesador.

2.2.9 Ámbito de reemplazo

La estrategia del ámbito de reemplazo se puede clasificar en global y local. Ambos tipos de políticas se activan por medio de un fallo de página cuando no existen marcos de página libres. La política de reemplazo local selecciona únicamente entre las páginas residentes del proceso que ha generado el fallo de página, mientras que en la política de reemplazo global se consideran todas las páginas en la memoria principal que no se encuentren bloqueadas como candidatos para el reemplazo, independientemente de a que procesos pertenezca cada página en particular. Mientras que las políticas locales son más fáciles de analizar, no existe ninguna evidencia que diga que proporciona un mejor rendimiento que las políticas globales.

Existe una correlación entre el ámbito de reemplazo y el tamaño del conjunto residente. Un conjunto residente fijo implica automáticamente una política de reemplazo local: para mantener el tamaño de conjunto residente, al reemplazar una página se debe eliminar de la memoria principal otra el mismo proceso. Una política de asignación variable puede emplear claramente la política de reemplazo global: el reemplazo de una página de un proceso en la memoria principal por la de otra causa que la asignación de memoria para un proceso crezca en una página mientras que disminuye la del otro.

2.2.10 Asignación fija, ámbito local

En este caso, se parte de un proceso que se encuentra en ejecución en la memoria principal con un numero de marcos fijo. Cuando se da un fallo de página, el sistema operativo debe elegir una página entre las residentes del proceso actual para realizar el reemplazo.

Con la política de asignación fija, es necesario decidir por adelantado la cantidad de espacio reservado para un proceso determinado. Esto se puede hacer en base al tipo de aplicación y al tamaño del programa. Las desventajas de esta estrategia son de dos tipos: si las reservas resultan ser demasiado pequeñas, va a haber una alta tasa de fallos de página, haciendo que el sistema multiprogramado completo se ralentice. Si las reservas son demasiado grandes, habrá muy pocos programas en la memoria principal y habrá mucho tiempo del procesador ocioso o mucho tiempo perdido en swapping.

Esta combinación es, probablemente, la más sencilla de implementar y ha sido adoptada por un gran número de sistemas operativos. En un momento determinado, existe un numero de procesos determinado en la memoria principal, cada uno de los cuales tiene una serie de marcos asignados. Normalmente, el sistema operativo también mantiene una lista de marcos libres. Cuando sucede un fallo de página, se añade un marco libre al conjunto residente de un

proceso y se trae la página a dicho marco. De esta forma, un proceso que sufra diversos fallos se página crecerá gradualmente en tamaño, lo cual debería reducir la tasa de fallos de página global del sistema.

	Reemplazo Local	Reemplazo Global
Asignación Fija	• El número de marcos asignados a un proceso es fijo. • Las páginas que se van a reemplazar se eligen entre los marcos asignados al proceso.	• No es posible
Asignación Variable	• El número de marcos asignados a un proceso pueden variarse de cuando en cuando. • Las páginas que se van a reemplazar se eligen entre los marcos asignados al proceso.	• Las páginas que se van a reemplazar se eligen entre todos los marcos de la memoria principal esto hace que el tamaño del conjunto residente de los procesos varíe.

Tabla 4: Gestión del conjunto residente

La dificultad de esta estrategia se encuentra en la elección de los reemplazos cuando no existen marcos libres disponibles, el SO debe elegir una página que actualmente se encuentra en la memoria para reemplazarla. Esta selección se lleva a cabo entre todos los marcos que se encuentran en la memoria principal, a excepción de los marcos bloqueados. Utilizando cualquiera de las políticas vistas en la sección anterior, se tiene que la página seleccionada para reemplazo puede pertenecer a cualquiera de los procesos residentes; nos existe ninguna disciplina predeterminada que indique que proceso debe perder una página de su conjunto residente.

Una forma de encontrar una contraprestación a los problemas de rendimiento potenciales de la asignación variable con reemplazo de ámbito global, se centran en el uso de buffering de páginas. De esta forma, la selección de una página para que se reemplace no es tan significativa, debido a que la página se puede reclamar si se hace referencia antes de que un nuevo bloque de páginas se sobrescriba.

2.2.12 Asignación variable, ámbito local

La asignación variable con reemplazo de ámbito local intenta resolver los problemas de la estrategia de ámbito global. Se puede resumir en lo siguiente:

1. Cuando se carga un nuevo proceso en la memoria principal, se le asigna un cierto número de marcos de página a su conjunto residente, basando en el tipo de aplicación, solicitudes del programa u otros criterios. Para cubrir esta reserva se utilizará la paginación adelantada o por demanda.

2. Cuando ocurra un fallo de página, la página que se seleccionará para reemplazar pertenecerá al conjunto residente del proceso que causo el fallo.

3. De vez en cuando, se reevaluará la asignación proporcionada a cada proceso, incrementándose o reduciéndose para mejorar al rendimiento.

Con esta estrategia, las decisiones relativas a aumentar o disminuir el tamaño del conjunto residente se toman de forma más “meditada” y se harán contando con los indicios sobre posibles demandas futuras de los procesos que se encuentran activos. Debido a la forma en la que se realiza esta valoración, esta estrategia es mucho más compleja que la política de reemplazo global simple. Sin embargo, puede llevar a un mejor rendimiento.

Los elementos claves en la estrategia de asignación variable con ámbito local son los criterios que se utilizan para determinar el tamaño del conjunto residente y la periodicidad de estos cambios. Una estrategia que ha sido llamativa es la estrategia del conjunto de trabajo. A pesar de que la estrategia del conjunto de trabajo pura sería difícil de implementar, es muy útil examinarla como referencia para las comparativas.

El conjunto de trabajo es un concepto acuñado y popularizado por Denning, ha tenido un profundo impacto en el diseño de la gestión de memoria virtual. El conjunto de trabajo con parámetro para un proceso en tiempo virtual es el conjunto de páginas del proceso a las que se ha referenciado en las últimas unidades de tiempo virtual.

El tiempo virtual se define de la siguiente manera. Considere la secuencia de referencias a memoria r (1), r (2) … en las cuales r(i) es la página que contiene la i-esima dirección virtual generada por dicho proceso. El tiempo se mide en referencias a memoria; así r=1,2,3… mide el tiempo virtual de un proceso.

Se considera que cada una de las dos variables de W. La variable (delta) es la ventana de tiempo virtual a través de la cual se observa al proceso. El tamaño del conjunto de trabajo será una función nunca decreciente del tamaño de ventana. El resultado que se encuentra en la Figura 8.19 demuestra la secuencia de referencias a paginas para un proceso. Los puntos indican unidades de tiempo en las cuales el conjunto de trabajo no cambia. Nótese que, para mayor tamaño ventada, el tamaño del conjunto de trabajo también es mayor.

El conjunto de trabajo también es una función del tiempo. Si un proceso ejecuta durante (delta) unidades de tiempo, y terminando el tiempo utiliza una única página, entonces, un

conjunto trabajo también puede crecer hasta llegar a las N páginas del proceso si se accede rápidamente a muchas páginas diferentes y si el tamaño de la ventana lo permite.

La figura 8.20 indica como puede variar el tamaño del conjunto de trabajo a lo largo del tiempo para un valor determinado de (delta). Para muchos programas, los periodos relativamente estables del tamaño de su conjunto de trabajo se alternan con periodos de cambio rápido. Cuando un proceso comienza a ejecutar, de forma gradual construye su conjunto de trabajo a medida que hace referencias a nuevas páginas. Esporádicamente, debido al principio de proximidad de referencia, el proceso deberá estabilizarse sobre un conjunto determinado de páginas. Los periodos transitorios posteriores reflejan el cambio del programa a una nueva región de referencia. Durante la fase de transición algunas páginas del antiguo conjunto de referencia permanecerán dentro de la ventana, causando un rápido incremento del tamaño del conjunto de trabajo a medida que se van referenciando nuevas páginas. Mientras la ventana se desplaza de estas referencias, el tamaño de conjunto de trabajo se reduce hasta que contengan únicamente aquellas páginas de la nueva región referenciada.

Este concepto del conjunto de trabajo se puede usar para crear la estrategia del tamaño conjunto residente:

1. Monitorizando el conjunto de trabajo de cada proceso.
2. Eliminando periódicamente del conjunto de residente aquellas páginas que no se encuentran en el conjunto de trabajo (LRU).
3. Un proceso puede ejecutar solo si su conjunto de trabajo se encuentra en la memoria principal.

Secuencia de referencias a páginas	Tamaño de ventana, Δ			
	2	3	4	5
24	24	24	24	24
15	24 15	24 15	24 15	24 15
18	15 18	24 15 18	24 15 18	24 15 18
23	18 23	15 18 23	15 18 23	15 18 23
24	23 24	18 23 24	•	•
17	24 17	23 24 17	18 23 24 17	15 18 23 24 17
18	17 18	24 17 18	•	18 23 24 17
24	18 24	•	24 17 18	•
18	•	18 24	•	24 17 18
17	18 17	24 18 17	•	•
17	17	18 17	•	•
15	17 15	17 15	18 17 15	24 18 17 15
24	15 24	17 15 24	17 15 24	•
17	24 17	•	•	17 15 24
24	•	24 17	•	•
18	24 18	17 24 18	17 24 18	15 17 24 18

Figura 13 Conjunto de trabajo de un proceso definido por el tamaño de ventana.

Esta estrategia funciona debido a que parte del principio de proximidad de referencia lo explota para conseguir una estrategia de la gestión de la memoria que minimice los fallos de página. Desgraciadamente, persisten varios problemas en la estrategia del conjunto de trabajo:

1. El pasado no siempre predice al futuro. Tanto el tamaño como la pertenencia al conjunto de trabajo cambian a lo largo del tiempo.
2. Una medición verdadera del conjunto de trabajo para cada proceso no es practicable. Sería necesario, por cada proceso, asignar un sello de tiempo a cada referencia a una página que asigne el tiempo virtual del proceso y que mantenga una lista ordenada por tiempo de las páginas de cada uno de ellos.
3. El valor optimo de (delta) es desconocido y en cualquier caso puede variar.

Sin embargo, el espíritu de esta estrategia si es válido, el gran número de sistemas operativos intentan aproximarse a la estrategia del conjunto de trabajo. La forma de hacer esto es centrarse no e las referencias exactas a páginas y si en la tasa de fallos de página. La tasa de fallos de página cae a medida que se incrementa el tamaño del conjunto residente de un proceso. El tamaño del conjunto de trabajo debe caer en un punto. Por tanto, en lugar de monitorizar el tamaño del conjunto de trabajo de forma directa se pueden alcanzar resultados comparables monitorizando la tasa de fallos de página. El razonamiento es el siguiente: si la tasa de fallos de

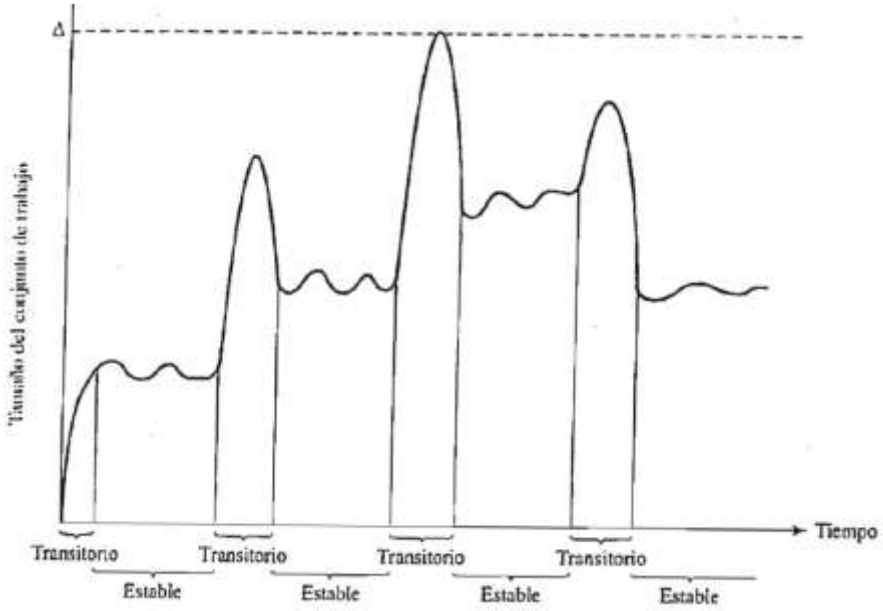


Figura 14 Grafico típico del tamaño del conjunto de trabajo

página de un proceso está por debajo de un determinado limite, el sistema de forma global se puede beneficiar de asignar un tamaño de conjunto residente menor para este proceso sin dañar al proceso en cuestión. Si la tasa de fallos de páginas entonces está por encima de un umbral máximo, el proceso puede beneficiarse de un incremento en el tamaño de su conjunto residente sin degradar el sistema.

Un algoritmo que sigue esta estrategia es el algoritmo de “frecuencia de fallos de página”. El algoritmo necesita un bit usado que se encuentre asociado a cada página de memoria. Este bit se pondrá a 1 cuando se haya accedido a la página. Cuando se produce un fallo de página, el SO anotara el tiempo virtual desde el ultimo fallo de página para dicho proceso; esto se puede realizar manteniendo un contador de las referencias a páginas. Se fina un umbra F. Si la diferencia de tiempo con el ultimo fallo de página es menor que este, entonces se añade una página al conjunto residente del proceso. En otro caso, se descartan todas las páginas con el bit de usado a 0, y se reduce el tamaño del conjunto residente de forma acorde. Mientras tanto, se pone a 0 el bit de usado del resto de las páginas. Esta estrategia se puede refinar usando dos umbrales: un umbral máximo que se utilizar para disparar el crecimiento del conjunto residente, y un límite inferior que se utilizar para disparar la reducción del tamaño del conjunto residente.

El tiempo entre fallos de página es reciproco a la tasa de fallos de página. A pesar de ello parecería mejor mantener una medida a lo largo de la ejecución de la tasa de fallos de página, sin embargo, la utilización de una medida de tiempo sencilla es un compromiso razonable que permite que las decisiones relativas al tamaño del conjunto residente se basen en la tasa de

fallos de página. Si una estrategia de este estilo se complementa con el buffering de páginas, se puede conseguir como resultado un óptimo rendimiento.

Sin embargo, existe un fallo grave en la estrategia adoptada por PFF, que hace que su comportamiento no sea bueno durante los periodos transitorios cuando se produce un desplazamiento hacia una nueva región referenciada. Con PFF ninguna página sale del conjunto de residente antes de que hayan pasado F unidades de tiempo virtual desde su última referencia. Durante estos periodos entre dos regiones de referencia, la rápida sucesión de fallos de página hace que el conjunto residente del proceso crezca antes de que las páginas de la antigua región de referencia se expulsen.

Una estrategia que intenta manejar este fenómeno de transición entre regiones de referencia con una sobrecarga relativamente baja comparado con PFF es la política de conjunto de trabajo con muestreo sobre intervalos variables (VSWS). La política VSWS evalúa el conjunto de trabajo del proceso en instantes de muestreo basados en el tiempo virtual transcurrido. Al comienzo del intervalo del muestreo, los bits de usado de las páginas residentes de procesos se ponen a 0; al final, solo las páginas a las que se ha hecho referencia durante el intervalo mantendrán dicho bit a 1; estas páginas se mantienen dentro del conjunto de residente del proceso a lo largo del siguiente intervalo, mientras que las otras se descartan. De esta forma el tamaño del conjunto residente solamente decrecerá al final del intervalo. Durante cada intervalo, todas las páginas que han causado fallo se añaden al conjunto residente; de esta forma el conjunto de residente mantiene un tamaño fijo o crece durante el intervalo.

La política VSWS toma tres diferentes parámetros:

- M: duración mínima del intervalo de muestreo.
- L: duración máxima del intervalo de muestreo.
- Q: número de fallos de página que se permite que ocurran entre dos instantes de muestreo.

La política VSWS es la siguiente:

1. Si el tiempo virtual entre el ultimo muestreo alcanza L, se suspende el proceso y se analizan los bits de usado.
2. Si, antes de que el tiempo virtual transcurrido llegue a L, ocurren Q fallos de página:

a. Si el tiempo virtual desde el ultimo muestreo es menor que M, se espera hasta que el tiempo virtual transcurrido alcance dicho valor para suspender el proceso y analizar los bits de usado.

b. Si el tiempo virtual desde el ultimo muestreo es mayor o igual a M, se suspende el proceso y se analizan los bits de usado.

Los valores de los parámetros se toman de forma que el muestreo se dispare habitualmente cuando ocurre el fallo de página Q después del último muestreo (caso 2b). Los otros dos parámetros (M y L) proporcionan fronteras de protección para condiciones excepcionales. La política VSWS intenta reducir el pico de solicitudes de memoria causadas por una transmisión abrupta entre dos áreas de referencia.

2.2.13 Política de limpieza

La política de limpieza es la opuesta a la política de recuperación: se encarga de determinar cuándo una página que esta modificada se debe escribir en memoria secundaria. Las dos alternativas más comunes son la limpieza bajo demanda y la limpieza adelantada. Con la limpieza bajo demanda, una página se escribe a memoria secundaria solo cuando se ha seleccionado para un reemplazo. En la política de la limpieza adelantada se escribe las páginas modificadas antes de que sus marcos de páginas se necesiten, de forma que las páginas se puedan escribir en lotes.

Existe un peligro en perseguir cualquiera de estas dos políticas hasta el extremo. Con la limpieza adelantada, una página se escribe, aunque continue en la memoria principal hasta que el algoritmo de reemplazo de páginas indique que debe eliminarse. La limpieza adelantada permite que las páginas se escriban en lotes, pero tiene poco sentido escribir cientos o miles d paginas para darnos cuenta de que la mayoría de ellas van a modificarse de nuevo antes de que sean reemplazadas. La capacidad de transferencia de la memoria secundaria es limitada y no se debe malgastar con operaciones de limpieza innecesarias.

Una estrategia más apropiada incorpora buffering páginas. Esto permite adopta la siguiente política: limpiar solo las páginas que son reemplazables, pero desacoplar las operaciones de limpieza y reemplazo. Con buffering de páginas, las páginas reemplazadas pueden ubicarse en dos listas: moduladas y no modificadas. Las que están en la lista de modificadas puede escribirse periódicamente por lotes y moverse después a la lista de no modificadas. Una página en la liste de no modificadas puede, o bien ser reclamada si se referencia, o perderse cuando su marco se asigna a otra página.

2.2.14 Control de carga

El control de carga determina el número de procesos que residirán en la memoria principal. La política de este control es crítica para una gestión de memoria efectiva. Si hay muchos procesos residentes a la vez, habrá muchas ocasiones en las cuales todos los procesos se encuentren bloqueados, y gran parte del tiempo se gastarán realizando swapping. Por otro lado, si hay demasiados procesos residentes, entonces, de media, el tamaño de conjunto residente de cada proceso será poco adecuado y se producirán frecuentes fallos de página.

2.2.15 Grado de multiprogramación

A medida que el nivel de multiprogramación aumente desde valores muy pequeños, cabría esperar que la utilización del procesador aumente, debido a que hay menos posibilidades de que los procesos residentes se encuentren bloqueados. Sin embargo, se alcanza un punto en el cual el tamaño de conjunto residente promedio no es adecuado. En este punto, el número de fallos de páginas se incrementa de forma dramática, y la utilización del procesador se colapsa.

Existen numerosas formas de abandonar este problema. Un algoritmo del conjunto de trabajo o de frecuencia de fallos de página incorporan, de forma implícita, control de carga. Solo aquellos procesos cuyo conjunto residente es suficientemente grande se les permite ejecutar.

Otra estrategia, se conoce como el criterio L = S, que ajusta el nivel de multiprogramación de forma que el tiempo medio entre fallos de página se iguala al tiempo medio necesario para procesar un fallo de página. Los estudios sobre el rendimiento indican que este es el punto en el

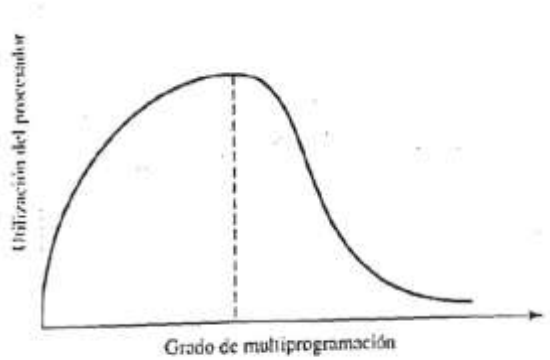


Figura 8.21. Efectos de la multiprogramación.

Figura 15 : efectos de la multiprogramación

cual la utilización del procesador es máxima. Una política que presenta un efecto similar es el criterio del 50%, que intenta mantener la utilización del dispositivo de paginación

aproximadamente al 50%. De nuevo, los estudios sobre el rendimiento indican que es el punto para el cual la utilización del procesador es máxima.

Otra alternativa es adaptar el algoritmo de reemplazo de páginas del reloj. Describe la técnica, usando un ámbito global, que implica monitorizar la tasa a la cual el puntero recorre el buffer circular de marcos. Si la tasa está por debajo de un nivel de umbral determinado, este indica alguna, o las dos, circunstancias siguientes:

1. Si están ocurriendo pocos fallos de página, que implican pocas peticiones para avanzar este puntero.
2. Por cada solicitud, el número de marcos promedio que se recorren por el puntero es pequeño, que indica que hay muchas páginas residentes a las que no se hace referencia y que son realmente reemplazables.

En ambos casos, el grado de multiprogramación puede incrementarse con seguridad. Por otro lado, si la tasa de recorrido circular del puntero supera un umbral máximo, indica que la tasa de fallos de páginas es alta o que hay dificultad para encontrar paginas reemplazables, lo cual implica que el grado de multiprogramación es demasiado alto.

2.2.16 Suspensión de procesos

Si se va a reducir el grado de multiprogramación, uno o más de los procesos actualmente residentes deben suspenderse, se proporcionan seis posibilidades:

- Procesos con baja prioridad: esto implementa una decisión de la política de activación y no se encuentra relacionadas con cuestiones de rendimiento.
- Procesos que provoca muchos fallos: la razón es que hay una gran probabilidad de que la tarea que causa los fallos no tenga su conjunto de trabajo residente, y el rendimiento sufrirá menos si dicha tarea se suspende.
- Proceso activado hace más tiempo: este es el proceso que tiene menor probabilidad de tener su conjunto de trabajo residente.
- Proceso con el conjunto residente de menor tamaño: este requerirá un menor esfuerzo para cargarse de nuevo. Sin embargo, penaliza a aquellos programas con una proximidad de referencias fuertes.
- Proceso mayor: ese proporciona un mayor número de marcos libres en una memoria que se encuentra en sobrecarga.

- Proceso con la mayor ventana de ejecución restante: en la mayoría de los casos, un proceso solo puede ejecutarse durante una determinada parte de tiempo antes de recibir la interrupción e irse al final e la lista de listos.

Es como en otras muchas áreas del diseño de sistemas operativos, la selección de la política más apropiada es una cuestión que considerar y depende de muchos otros factores de diseño.

2.3 Gestión de memoria de Unix y solaris

Debido a que UNIX pretende ser un sistema independiente de la máquina, su esquema de gestión de memoria variará en un sistema a otro. En las primeras versiones, UNIX particionamiento variable sin ningún esquema de memoria virtual. Actualmente Unix y Solaris utilizan la memoria virtual paginada.

En SVR4 y Solaris, existen dos esquemas de gestión de memoria separados. El sistema de paginación proporciona las funcionalidades de la memoria virtual para asignar marcos de página en la memoria principal de los diferentes procesos y también asignar marcos de página a buffers de bloques de disco. A pesar de que este es un esquema efectivo para los procesos de usuario y la E/S de disco, un esquema de la memoria virtual paginada es menos apropiado para gestionar la asignación de memoria del núcleo. Por esas cuestiones, se utiliza un asignador de memoria del núcleo.

2.3.1 Sistema de paginación

Estructuras de datos. Para la memoria virtual paginada, UNIX utiliza varias estructuras de datos que, con pequeñas diferencias, son independientes de la máquina (Figura 8.22 y Tabla 8.5).

- Tabla de páginas. Habitualmente, habrá una tabla de páginas por proceso, con una entrada por cada página de memoria virtual del proceso.
- Descriptor de bloques de disco. Asociado a cada página de proceso hay una entrada en esta tabla que indica la copia en disco de la página virtual.
- Tabla de datos del marco de página. Describe cada marco de memoria real y se indexa por medio de un número de marco. El algoritmo de reemplazo usa esta tabla.
- Tabla de utilización de swap. Existe una tabla de uso de swap por cada dispositivo de intercambio, con una entrada por cada página de dicho dispositivo.



Figura 16: Formatos de gestión de memoria de UNIX SVR4

La mayoría de los campos definidos en la Tabla 8.5 proporcionan su propia descripción. Unos pocos de ellos requieren una explicación adicional. El campo Edad es un indicador de cuánto tiempo hace que el programa no ha hecho referencia a este marco. El número de bits y la frecuencia de actualización de este campo son dependientes de la implementación.

El campo Tipo de Almacenamiento se necesita por la razón de: cuando un fichero ejecutable se usa por primera vez para crear un nuevo proceso, solo parte del programa y de los datos de ficho fichero se cargan en la memoria real. Mas tarde según se van dando fallos de página, nuevas partes del programa de los datos se irán cargando. En el momento de la carga inicial es cuando se crean las páginas de memoria virtual y se las asocia con posiciones en un dispositivo que se usa para ello. En ese momento el S.O indica si es necesario limpiar (poner a 0) las posiciones en el marco de página antes de la primera carga de un bloque de programa o datos.

Reemplazo de páginas. La tabla de marcos de páginas se utiliza para el reemplazo de las páginas. Todos los marcos disponibles se enlazan en una lista de marcos libres disponibles para traer páginas. Cuando el número de marcos disponibles es menor a un determinado nivel, el núcleo quitara varios marcos para compensar.

El algoritmo de reemplazo de páginas usado en SVR4 es un refinamiento del algoritmo de reloj, conocido como algoritmo de reloj con dos manecillas. Utiliza bit de referencia que se coloca en 0 o 1 según fue referenciada la página para lectura o escritura. La manecilla

delantera recorre las páginas de las seleccionables y pone el bit a 0 para cada una. Luego la manecilla trasera recorre y verifica el bit de referencia. Si está en 1 solamente se salta y si está en 0 la coloca en la lista de páginas expulsable.

Dos parámetros determinan la operación de este algoritmo:

- La tasa de recorrido: tasa a la cual las dos manecillas recorren la lista de páginas, en paginas por segundo.
- La separación entre manecillas: el especio entre manecillas delantera y trasera.

A medida que la memoria libre se reduce, las manecillas se mueven más rápido para liberar más páginas. El parámetro de separación entre manecillas junto con la tasa de recorrido indica la ventana de oportunidad para usar una página antes de que sea descartable por falta de uso.

Asignador de memoria del núcleo. El núcleo genera y destruye con frecuencia procesos pequeños y tablas de diversos tipos. Para ello, utiliza un conjunto de algoritmos de gestión de memoria dinámico [VAH496] muestra varios ejemplos:

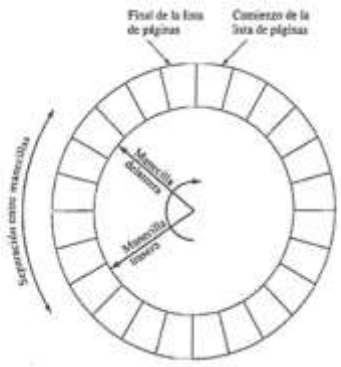


Figura 17: Algoritmos de reemplazo de páginas de reloj de dos manecillas

Entrada de la tabla de páginas	
Número de marco de página	Indica el marco de la memoria real.
Edad	Indica cuánto tiempo ha pasado la página en la memoria sin haberse referenciado. La longitud y contenidos de este campo dependen del procesador.
***Copy on write*[8]**	Puesto a 1 cuando más de un proceso comparte esta página. Si uno de los procesos escribe en esta página, se debe hacer primero una copia aparte de la misma para todos los procesos que aún comparten la página original. Esta funcionalidad permite demorar la operación de copia hasta que sea completamente necesaria y evitar los casos en los cuales no llega a serlo.
Modificada	Indica si la página se ha modificado.
Referenciada	Indica que se ha hecho referencia a esta página. Este bit se pone a 0 cuando la página se carga por primera vez y se puede reiniciar periódicamente por parte del algoritmo de reemplazo de páginas.
Válida	Indica si la página se encuentra en la memoria principal.
Protegida	Indica si se permite la operación escritura o no.
Descriptor de bloques de disco	
Número de dispositivo de *swap*	Número de dispositivo lógico de almacenamiento secundario que almacena la página correspondiente. Se permite que existan más de un dispositivo para ser utilizados como *swap*.
Número de bloque del dispositivo	Posición del bloque de la página en el dispositivo de *swap*.
Tipo de almacenamiento	El dispositivo de almacenamiento puede ser una unidad de *swap* o un fichero ejecutable. En el último caso, existe una indicación que denota si la memoria virtual asignada debe borrarse o no.
Entrada en la tabla de marcos de página	
Estado de la página	Indica si este marco se encuentra disponible o si está asociado a una página. En este último caso, especifica si la página se encuentra en dispositivo de *swap*, en fichero ejecutable, o en una operación de E/S.
Contador de referencias	Número de procesos que hacen referencia a esta página.
Dispositivo lógico	Dispositivo lógico que contiene una copia de la página.
Número de bloque	Posición del bloque de la copia de la página sobre el dispositivo lógico.
Punteros datos MP	Puntero a otros datos que apuntan a otras entradas en la tabla de marcos de página. Con este puntero se puede construir la lista de páginas libres o una lista *hash* de páginas.
Entrada en la tabla de uso de *swap*	
Contador de referencias	Número de entradas en la tabla de página que apunta a esta página en el dispositivo de *swap*.
Número de la unidad de almacenamiento/página	Identificador de la página en la unidad almacenamiento

Tabla 3: parámetros de gestión de memoria de UNIX

- El encaminamiento para traducción de una ruta puede reservar un buffer para copiar la ruta desde el espacio de usuario.
- La rutina a11ocb) reserva un buffer para tamaño arbitrario.

- Muchas implementaciones de UNIX reservan estructuras zombis para guardar el estado de salida de un proceso muerto.
- En SVR4 y Solaris, el núcleo reserva muchos objetos (como estructuras del proc, vnodos, y bloques de descriptores de fichero) de forma dinámica y bajo demanda.

La mayoría de sus bloques son realmente más pequeños que el tamaño típico de una página de la máquina. Por lo tanto, el mecanismo de paginación sería muy deficiente a la hora de reservar memoria dinámica del núcleo. SVR4 usa modificación del sistema buddy.

En los sistemas buddy, el coste de reservar y liberar un bloque de la memoria es bajo comparado con políticas de mejor ajusta y primer ajuste. La desventaja de los sistemas buddy

Barkley y Lee de AT&T propusieron el sistema buddy perezoso una variación del sistema buddy. Los autores observaron que UNIX muestra comportamiento estable cuando la cantidad de peticiones para bloques de un determinado tamaño varia ligeramente a lo largo del tiempo. Por tanto, si un bloque del tamaño 2^i se libera y se agrupa con su vecino en bloque de tamaño 2^i+1, cuando el núcleo solicite un bloque de tamaño 2^i, implica la necesidad de dividir el bloque de mayor tamaño de nuevo. Para evitar esta agrupación innecesaria y su posterior división. El sistema buddy perezoso pospone la reagrupación hasta el momento en el que parezca que resulta necesaria y en ese momento intenta reagrupar el mayor número de bloques posibles.

El sistema buddy perezoso usa los siguientes parámetros:

- N = número actual de bloques de tamaño 2^i.
- A = número actual de bloques de tamaño 2^i que se encuentran reservados (ocupados).
- G = número actual de bloques de tamaño 2^i que se encuentran globalmente libres. Estos son los bloques que se pueden seleccionar para ser reagrupados.
- L = número actual de bloques de tamaño 2^i que se encuentran localmente libres. Estos bloques son los que no se encuentran seleccionables para su grabación. No se reagrupa con su vecino.

La situación relativa se mantiene:

- Ni = Ai + Gi + Li

En general, el sistema buddy perezoso intenta mantener un caudal de bloques libres locales y únicamente solicita la reagrupación si el número de bloques libres locales supera un determinado limite. Si hay muchos bloques libres locales, entonces existe la posibilidad de que falten bloques libres en el siguiente nivel. La mayoría del tiempo, cuando se libera un bloque, la reagrupación no se realiza de forma que se minimizan los costes de gestión y de operaciones.

El criterio que se utiliza para la reagrupación es que el número de bloques libres locales del tamaño determinado no puede exceder el número de bloques reservados para ese tamaño (Li >= Ai).

Para implementar este esquema, los autores de definen una variable de demora de la siguiente forma:

- Di = Ai - Li = Ni – 2Li - Gi

2.3.2 Gestión de memoria en Linux

Linux comparte muchas de las características de los esquemas de gestión de la memoria de otras implementaciones de UNIX, pero incorpora sus propias características. Por encina de todo, el esquema de gestión de la memoria de Linux es bastante complejo.

2.3.3 Memoria virtual en Linux

Direccionalmente de la memoria virtual. Linux usa una estructura de tablas de páginas de tres niveles consistente en los siguientes tipos de tablas (cada tabla en particular tiene el tamaño de una página):

- Directorio de páginas. Un proceso activo tiene un directorio de páginas único que tiene el tamaño de una página. Cada entrada en el directorio de páginas apunta a una página en el directorio intermedio de páginas. El directorio de páginas debe residir en la memoria principal para cualquier proceso activo.
- Directorio intermedio de páginas. El directorio intermedio de páginas se expande a múltiples páginas. Cada entrada en el directorio intermedio paginas apunta a una página que contiene una tabla de páginas.
- Tabla de páginas. La tabla de páginas también puede expandirse a múltiples páginas. Cada entrada en la tabla de páginas hace referencia a una página virtual del proceso.

Una dirección virtual en Linux se puede ver como consistente en cuatro campos (Figura 8.25), el campo de más a la izquierda se utiliza como índice en el directorio de páginas. El siguiente campo como índice en el directorio intermedio de páginas. El tercer campo se utiliza para indexar en la tabla de páginas. Y el cuarto campo se proporciona el desplazamiento dentro de la página de la memoria seleccionada.

Reserva de páginas. Para mejorar la eficiencia de lectura y escritura de páginas en la memoria principal, Linux define un mecanismo para manejar bloques de páginas contiguas que se proyectarán sobre bloques de marcos de página también contiguos. Con este fin, también se utiliza el sistema buddy.

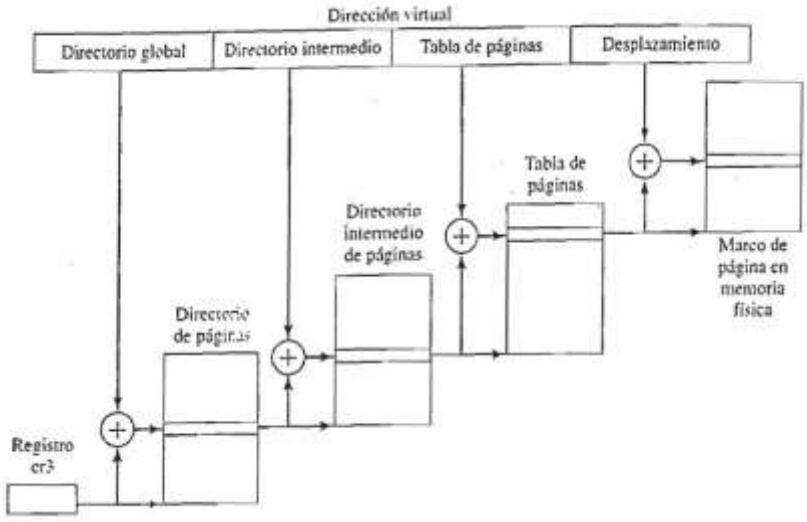


Figura 18: traducción de direcciones en el esquema de memoria virtual en de Linux.

Algoritmo de reemplazo de páginas. El algoritmo de reemplazo de páginas de Linux se basaba en el algoritmo del reloj. Pero se puede decir que es una variante de la política LRU.

2.3.4 Reserva de memoria del núcleo

La gestión de la memoria del núcleo se realiza en base a los marcos de página de memoria principal. Su función básica es asignar y liberar marcos para diferentes usos. Los posibles propietarios de un marco incluyen procesos en espacio de usuario, datos del núcleo reservados dinámicamente, y caché de páginas.

Se utiliza el algoritmo de buddy de forma que la memoria del núcleo se pueda reservar y liberar en unidades de una o más páginas. Debido a que el tamaño mínimo de memoria que se pueda reservar de esta forma es una página, la reserva de páginas únicamente sería insuficiente debido a que el núcleo requiere pequeños fragmentos que se utilizarán durante un corto periodo de tiempo y que son de diferentes tamaños. Para ajustarse a esos pequeños tamaños Linux utiliza un esquema complejo llamado asignación por laminas dentro de una página ya reservada.

2.4 Gestión de memoria en Windows

El gestor de memoria virtual en Windows controla la forma en la que se reserva la memoria y como se realiza la paginación. El gestor de memoria se ha diseñado para funcionar sobre una variada gama de plataformas y para utilizar tamaños de páginas que van desde los 4 Kbytes hasta los 64 Kbytes.

2.4.1 Mapa de direcciones virtuales en Windows

Cada proceso de usuario en Windows puede ver un espacio de direcciones independientes de 32 bits, permitiendo 4Gbytes de memoria por proceso. Una parte de esta memoria está reservada para el SO, de forma que el usuario dispone de 2Gbytes de espacio virtual de direcciones disponibles y todos los procesos comparten los mismos 2Gbytes de espacio del sistema. Existe una opción que permite que el espacio crezca hasta los 3Gbytes, dejando un espacio de sistema de únicamente 1 Gbytes. Esta característica se incluyó para dar soporte a aplicaciones que requieren uso intensivo de grandes cantidades de memoria en servidores con memorias de varios gigabytes.

La Figura 8.26 muestra el espacio de direcciones virtuales que, por defecto, ve un usuario. Consiste en cuatro regiones:

- 0x00000000 a 0x0000FFFF. Reservado para ayudar a los programadores a capturar asignaciones nulas.
- 0x00010000 a 0x7FFFFFFF. Espacio de direcciones disponible para el usuario. Este espacio se encuentra dividido en páginas que pueden cargarse en la memoria principal.
- 0x7FFF0000 a 0x7FFFFFFF. Una página de guarda, no accesible por el usuario. Esta página hace que sea más fácil para el sistema operativo verificar referencias a punteros fuera de rango.
- 0x80000000 a 0xFFFFFFFF. Espacio de direcciones del sistema. Esta área de 2 Gbytes se utiliza por parte del Ejecutivo de Windows, el micronúcleo y los manejadores de dispositivos.

2.4.2 Paginación en Windows

Cuando se crea un proceso, puede, en principio, utilizar todo el espacio de usuario de 2 Gbytes. Este espacio se encuentra dividido en páginas de tamaño fijo, cualquiera de las cuales se puede cargar en memoria principal. En la práctica existen los siguientes estados:

- Disponible. Páginas que no están actualmente usadas por este proceso.
- Reservada. Conjunto de páginas contiguas que el gestor de memoria virtual separa para un proceso pero que no se cuentan para la cuota de memoria usada por dicho proceso.
- Asignada. Las páginas para las cuales el gestor de la memoria virtual ha reservado espacio en el fichero de paginación.

La distinción entre la memoria reservada y asignada es muy útil debido a que (1) minimiza la cantidad de espacio en disco que debe guardarse para un proceso en particular, manteniendo espacio libre en disco para otros procesos; y (2) permite que un hilo en un proceso declare una petición de una cantidad de memoria que puede proporcionar rápidamente si se necesita.

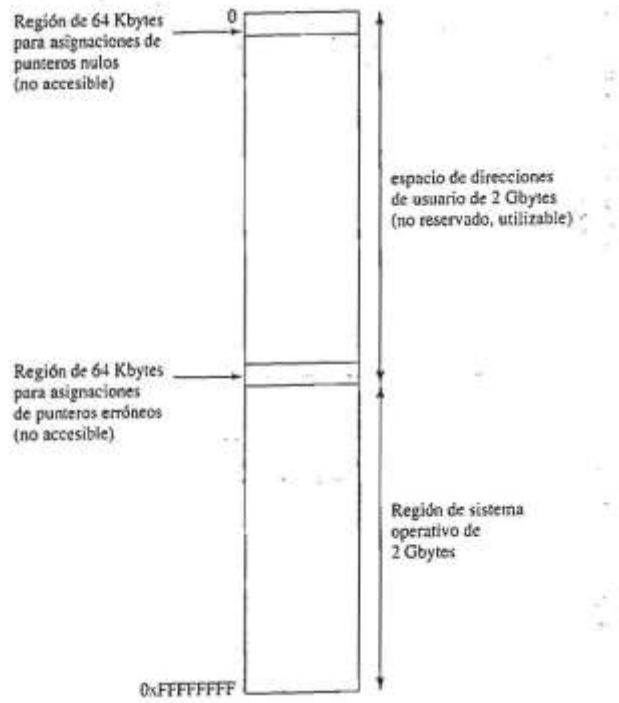


Figura 19: Espacio de direcciones virtuales habitual de Windows.

El esquema de gestión del conjunto residente que utiliza Windows es de asignación variable, con ámbito local. Cuando se activa un proceso por primera vez, se le asigna un cierto número de marcos de página de la memoria principal como conjunto de trabajo. Cuando un proceso hace referencia a una página que no está en memoria, una de las páginas residentes de dicho proceso se expulsa, trayéndose la nueva página.

3. CONCLUSIÓN

La memoria virtual representa uno de los avances más trascendentales en la arquitectura de los sistemas operativos, al resolver la limitación física de la memoria principal mediante una estrecha integración entre el hardware de soporte (como la MMU y la TLB) y las políticas de software del sistema operativo. A través de mecanismos como la paginación y la segmentación, sustentados sólidamente en el principio de proximidad de referencias, el sistema logra desacoplar el espacio de direccionamiento lógico de la capacidad real instalada. Esto no solo garantiza la ejecución fluida de procesos que superan el tamaño de la memoria RAM, sino que optimiza drásticamente el nivel de multiprogramación y la utilización de la CPU.

Asimismo, la eficiencia global del sistema depende de un equilibrio crítico en las políticas de gestión: la selección acertada de los tamaños de página, el reemplazo de marcos (mediante aproximaciones viables a LRU como el algoritmo del reloj), el control estricto del conjunto residente para prevenir el hiperpaginado (*thrashing*) y el uso de técnicas de *buffering*. Tal como se evidencia en las implementaciones analizadas de UNIX/Solaris, Linux y Windows, cada arquitectura adapta estos conceptos fundamentales —ya sea mediante tablas de múltiples niveles, esquemas de asignación dinámicos como *buddy system* o gestiones precisas del espacio de direcciones— para lograr un compromiso óptimo entre sobrecarga operativa, rendimiento de entrada/salida y aislamiento seguro entre procesos.

4. REFERENCIAS BIBLIOGRÁFICAS

[1] W. Stallings. Operating Systems: Internals and Design Principles, 5ta edición, Pearson, 2005.